大数据平台部署实战手册

从零基础到竞赛实战

竞赛培训组

2026 年 3 月 29 日

学习指引

前序课程：如果你还不熟悉 Python 数据处理，建议先学习《Pandas 数据处理实战手册》。
后续课程：平台部署好后，下一步是在平台上做数据分析。
配套练习：更多练习见仓库根目录的《仿真实战练习.md》。

目录

1 前言：为什么要学习大数据平台部署？

什么是大数据平台？

大数据平台是一套用于存储、处理和分析海量数据的软件系统集合。在竞赛中，大数据平台部署是考察学生实际操作能力的重要环节。

核心组件：

- **Hadoop**：分布式计算框架，包含 HDFS 和 YARN
- **HDFS**：分布式文件系统，用于存储海量数据
- **HBase**：分布式 NoSQL 数据库
- **Hive**：数据仓库工具，提供 SQL 查询能力
- **Spark**：快速通用的大数据计算引擎
- **Zookeeper**：分布式协调服务
- **Kafka**：高吞吐量消息队列
- **Flume**：数据采集工具
- **ClickHouse**：列式数据库，用于 OLAP 分析

竞赛中的大数据平台部署

在竞赛中，大数据平台部署部分要求参赛选手：

1. 根据题目要求完成大数据平台的部署
2. 包括但不限于 Hadoop、HDFS、HBase、Hive、Spark、Zookeeper、Flume、Kafka、ClickHouse 等
3. 能够根据要求对部署的平台参数进行调整
4. 测试平台可用性，对多个平台进行联调、联测
5. 实现数据在平台中的流转

2 一个月速成指南：理解每个组件

2.1 全局视角：数据是怎么流的？

一句话理解整个系统

大数据平台就是一个”数据加工厂”，数据从原材料（日志）到成品（统计报告）经过 6 道工序：

```
[日志文件] --> [Flume] --> [Kafka] --> [Spark] --> [HBase/CH] --> [Hive]
 原材料       搬运工      传送带      加工车间      成品仓库       质检报告
```

2.2 每个组件用一句话说清楚

表 1: 组件速查表

组件	一句话解释	类比	部署在哪
HDFS	分布式硬盘，切块存多台	拼图仓库	node1(NN), node2/3(DN)
YARN	调度中心，分配 CPU/内存	食堂阿姨	node1(RM), node2/3(NM)
Zookeeper	分布式班长，协调各组件	选班长	三台都装
HBase	按关键词快速查海量数据	通讯录	node1(HM), node2/3(HRS)
Hive	SQL 翻译器，查 HDFS 数据	标签系统	node1
Spark	内存计算，比 MR 快百倍	草稿纸	node1(M), node2/3(W)
Kafka	消息传送带，缓冲数据	传送带	node1
Flume	数据搬运工，日志到 Kafka	搬家公司	node1
ClickHouse	超级统计表，分析极快	超级 Excel	node2

2.3 它们之间怎么配合？

数据流转详解（考试必考）

场景：电商网站产生用户行为日志，需要实时统计每个用户的行为次数。

1. **数据产生**：用户在网站上点击、浏览、购买，操作被记录到日志文件
2. **Flume 采集**：Flume 的 Source 监控日志文件，发现新数据就读取出来
3. **Kafka 缓冲**：Flume 把数据发送到 Kafka Topic（名为 user-behavior），数据暂存在 Kafka 中
4. **Spark 处理**：Spark Streaming 从 Kafka 消费数据，实时统计每个用户每种行为的次数
5. **HBase 存储**：统计结果写入 HBase，方便按用户 ID 快速查询

6. **ClickHouse 分析**：统计结果也写入 ClickHouse，方便按品类、时间做统计分析

7. **Hive 离线查询**：Hive 把 HBase 中的数据映射成表格，用 SQL 做复杂查询

竞赛中的常见考法

- 考法 1：给定组件列表，让你按正确顺序启动（答案：ZK → HDFS → Kafka → 其他）
- 考法 2：给定数据流图，让你配置 Flume、Kafka、Spark 实现数据流转
- 考法 3：某个组件启动失败，让你排查原因（看日志！）
- 考法 4：调整参数（比如 HDFS 副本数、Kafka 分区数）并验证效果
- 考法 5：创建 Hive 表映射 HBase 数据，用 SQL 查询

3 环境准备

3.1 硬件与系统要求

表 2: 硬件配置要求

组件	最低配置	推荐配置
CPU	4 核	8 核 +
内存	8GB	16GB+
磁盘	50GB	100GB+
系统	CentOS 7/Ubuntu 18.04	CentOS 7/Ubuntu 20.04

3.2 组件版本兼容性

表 3: 组件版本兼容性参考（基于 3 节点 CentOS 7/Ubuntu 20.04）

组件	本手册版本	依赖组件	注意事项
JDK	8u212	-	所有组件共用
Hadoop	3.3.4	JDK 8+	
Zookeeper	3.7.1	JDK 8+	
HBase	2.4.15	HDFS + Zookeeper 3.5-3.7	
Hive	3.1.3	HDFS + MySQL 5.7+	需 MySQL JDBC 8.x 驱动
Spark	3.3.2	JDK 8+ + HDFS	预编译含 Hadoop 依赖
Kafka	3.4.0	Zookeeper	
Flume	1.11.0	Kafka + JDK 8+	
ClickHouse	22.x+	-	独立安装
MySQL	5.7+	-	Hive 元数据存储

重要

- Zookeeper 版本必须与 HBase 兼容：HBase 2.4.x 要求 Zookeeper 3.5-3.7
- Hive 与 Spark 联用时，需确保依赖包版本一致
- MySQL JDBC 驱动需使用 8.0+ 版本（如 mysql-connector-java-8.0.33.jar）

3.3 集群节点规划

一主两从集群规划（竞赛标准配置）

竞赛环境中，使用 3 台服务器组成集群。以下为完整的服务分配方案：

表 4: 节点角色与服务分配

主机名	IP	角色	部署服务
node1	192.168.1.101	Master	NameNode, SecondaryNameNode ResourceManager, HMaster HiveServer2, Spark Master Zookeeper, Kafka, Flume
node2	192.168.1.102	Worker	DataNode, NodeManager HRegionServer, Spark Worker Zookeeper, ClickHouse
node3	192.168.1.103	Worker	DataNode, NodeManager HRegionServer, Spark Worker Zookeeper

记忆口诀:

- **Master 服务**只装在 node1（NameNode、ResourceManager、HMaster）
- **Worker 服务**装在 node2 和 node3（DataNode、NodeManager）
- **Zookeeper** 三台都要装（需组成集群选 Leader）
- **Kafka** 只装在 node1（竞赛环境单 Broker 足够）
- **Flume** 装在 node1（和 Kafka 在同一台）

竞赛中的操作流程

在竞赛中，你会拿到 3 台已装好系统的服务器。操作顺序:

1. **基础环境**（本节）: 改主机名、配 hosts、关防火墙、装 JDK、配 SSH 免密
2. **逐个部署组件**（后续章节）: Zookeeper → Hadoop → HBase → Hive → Spark → Kafka → Flume → ClickHouse
3. **联调测试**（实战章节）: 启动数据流，验证端到端

重要: 每一步都标明了”在哪个节点上执行”:

- `#` 【在 node1 上执行】 = 只在这一个节点操作
- `#` 【所有节点】 = 三台都要执行

3.4 基础环境配置

3.4.1 修改主机名

```
# 在node1上执行
hostnamectl set-hostname node1

# 在node2上执行
hostnamectl set-hostname node2

# 在node3上执行
hostnamectl set-hostname node3
```

3.4.2 配置 hosts 文件

```
# 编辑 /etc/hosts，所有节点保持一致
vi /etc/hosts

# 添加以下内容
192.168.1.101 node1
192.168.1.102 node2
192.168.1.103 node3
```

注意

所有节点的 hosts 文件必须保持一致，否则组件间通信会失败！

3.4.3 关闭防火墙和 SELinux

```
# 关闭防火墙
systemctl stop firewalld
systemctl disable firewalld

# 关闭SELinux
setenforce 0
sed -i 's/SELINUX=enforcing/SELINUX=disabled/g' /etc/selinux/
    config
```

3.4.4 配置 SSH 免密登录

为什么要配置 SSH 免密？

Hadoop 启动时，node1 需要通过 SSH 远程在 node2、node3 上启动 DataNode。如果没有免密，每次都要输入密码，启动脚本就无法自动运行。

```
# 【在 node1 上执行】生成密钥对
ssh-keygen -t rsa -P '' -f ~/.ssh/id_rsa

# 【在 node1 上执行】把公钥发给所有节点（包括自己）
ssh-copy-id node1
ssh-copy-id node2
ssh-copy-id node3

# 【在 node1 上执行】测试免密登录
ssh node2 hostname  # 应显示 node2，不需密码
ssh node3 hostname  # 应显示 node3，不需密码
```

如果 ssh-copy-id 失败

```
# 在 node1 上读取公钥
cat ~/.ssh/id_rsa.pub

# 在 node2、node3 上手动添加
mkdir -p ~/.ssh && chmod 700 ~/.ssh
echo "ssh-rsa AAAA..." >> ~/.ssh/authorized_keys
chmod 600 ~/.ssh/authorized_keys
```

3.4.5 安装 JDK

```
# 上传JDK安装包并解压
tar -zxvf jdk-8u212-linux-x64.tar.gz -C /opt/

# 配置环境变量
vi /etc/profile

# 添加以下内容
```

```
export JAVA_HOME=/opt/jdk1.8.0_212
export PATH=$JAVA_HOME/bin:$PATH
export CLASSPATH=.:$JAVA_HOME/lib/dt.jar:$JAVA_HOME/lib/tools.jar

# 使配置生效
source /etc/profile

# 验证安装
java -version
```

动手练习

1. 完成 3 台服务器的 hosts 文件配置
2. 配置 SSH 免密登录，确保 node1 可以免密登录到 node2 和 node3
3. 安装 JDK 并验证版本

4 Hadoop 集群部署

4.1 Hadoop 是什么？

用快递仓库理解 Hadoop

想象你要管理一个巨大的快递仓库：

- 一个仓库放不下所有包裹 → 需要多个仓库（分布式存储 = HDFS）
- 多个仓库需要有人调度谁去哪个仓库取货（资源调度 = YARN）
- 取货的路线需要规划（计算框架 = MapReduce）

一句话总结：Hadoop 让你用很多台普通电脑协同工作，处理一台电脑处理不了的海量数据。

Hadoop 的三大核心组件

- **HDFS**（Hadoop Distributed File System）：分布式文件系统，把大文件切成小块分散存到不同机器上
- **YARN**（Yet Another Resource Negotiator）：资源调度器，决定哪个任务用

哪台机器的 CPU 和内存

- **MapReduce**:编程模型,把一个大计算任务拆成”分(Map)”和”合(Reduce)”两步并行执行

竞赛中你需要做的：在 node1 上配置 Hadoop，然后用启动脚本自动把 DataNode 分发到 node2、node3 上。

4.2 Hadoop 简介

Hadoop 核心组件

Hadoop 是一个开源的分布式计算框架，核心组件包括：

- **HDFS**：分布式文件系统，用于存储海量数据
- **YARN**：资源调度框架，负责集群资源管理
- **MapReduce**：分布式计算框架

4.3 下载与解压

```
# 下载Hadoop（以3.3.4版本为例）
# 如果从 PDF 复制 URL 有空格，手动删除空格后粘贴
wget https://repo.huaweicloud.com/apache/hadoop/common/hadoop
    -3.3.4/hadoop-3.3.4.tar.gz

# 解压到/opt目录
tar -zxvf hadoop-3.3.4.tar.gz -C /opt/

# 创建软链接
ln -s /opt/hadoop-3.3.4 /opt/hadoop
```

4.4 配置环境变量

```
vi /etc/profile

# 添加Hadoop环境变量
export HADOOP_HOME=/opt/hadoop
```

```
export PATH=$HADOOP_HOME/bin:$HADOOP_HOME/sbin:$PATH

# 使配置生效
source /etc/profile

# 验证
hadoop version
```

4.5 修改 Hadoop 配置文件

配置文件位于 $HADOOP_HOME/etc/hadoop/ 目录下。

4.5.1 hadoop-env.sh——告诉 Hadoop 用哪个 Java

这个文件是干什么的？

hadoop-env.sh 是 Hadoop 的”环境变量配置文件”。Hadoop 是用 Java 写的，所以它必须知道 Java 装在哪里。

类比：就像你在 Windows 上安装软件时，如果软件找不到安装路径，你就需要手动指定。这个文件就是告诉 Hadoop：”Java 装在 /opt/jdk1.8.0_212 这个目录下”。

```
# 打开配置文件
vi $HADOOP_HOME/etc/hadoop/hadoop-env.sh

# ====== 第1个配置：告诉Hadoop Java装在哪里 ======
# 如果不配置，Hadoop 会报错 "JAVA_HOME is not set"
export JAVA_HOME=/opt/jdk1.8.0_212
# 注意：这里要写你实际安装的 JDK 路径
# 查看方法：在终端执行 echo $JAVA_HOME 或 which java

# ====== 第2个配置：允许用root用户运行 ======
# Hadoop 默认不允许用 root 用户启动（出于安全考虑）
# 竞赛中通常用 root 用户，所以需要显式允许
export HDFS_NAMENODE_USER=root        # 允许root运行NameNode
export HDFS_DATANODE_USER=root        # 允许root运行DataNode
export HDFS_SECONDARYNAMENODE_USER=root  # 允许root运行
    SecondaryNameNode
export YARN_RESOURCEMANAGER_USER=root    # 允许root运行
    ResourceManager
```

```
export YARN_NODEMANAGER_USER=root          # 允许root运行 NodeManager
```

4.5.2 core-site.xml——告诉 Hadoop NameNode 在哪里

这个文件是干什么的？

core-site.xml 是 Hadoop 的”核心配置文件”，相当于 Hadoop 的”身份证”。
最重要的一个配置：fs.defaultFS——告诉所有组件”NameNode 在哪台机器上”。所有 HDFS 操作（上传、下载、读取文件）都要先问 NameNode：”这个文件存在哪里？”
类比：就像你去图书馆找书，首先要查”图书目录”（NameNode），目录告诉你书在哪个书架（DataNode）。

```xml
<?xml version="1.0" encoding="UTF-8"?>
<?xml-stylesheet type="text/xsl" href="configuration.xsl"?>
<configuration>
    <!-- ====== 配置1：NameNode地址（最重要！） ====== -->
    <!-- fs.defaultFS = "文件系统的默认地址" -->
    <!-- hdfs:// 表示使用 HDFS 协议 -->
    <!-- node1:9000 表示 NameNode 在 node1 的 9000 端口 -->
    <!-- 如果这个地址写错，整个 HDFS 就连不上！ -->
    <property>
        <name>fs.defaultFS</name>
        <value>hdfs://node1:9000</value>
    </property>

    <!-- ====== 配置2：Hadoop临时目录 ====== -->
    <!-- Hadoop 运行时产生的临时文件存放位置 -->
    <!-- 包括 NameNode 的元数据、DataNode 的数据块等 -->
    <!-- 建议放在空间大的磁盘分区 -->
    <property>
        <name>hadoop.tmp.dir</name>
        <value>/opt/hadoop/tmp</value>
    </property>

    <!-- ====== 配置3：Web界面用户 ====== -->
    <!-- 访问 HDFS Web 界面（http://node1:9870）时显示的用户名 -->
```

```
    <!-- 设为 root 方便查看 -->
    <property>
        <name>hadoop.http.staticuser.user</name>
        <value>root</value>
    </property>
</configuration>
```

4.5.3 hdfs-site.xml——配置 HDFS 存储策略

这个文件是干什么的？

hdfs-site.xml 是 HDFS（分布式文件系统）的专用配置文件。它告诉 Hadoop：

- 文件存在哪里（namenode 目录、datanode 目录）
- 每个数据块存几份（副本数）
- SecondaryNameNode 在哪

最重要的是副本数：默认 3 份。比如你上传一个文件，HDFS 会自动在 3 台机器上各存一份，这样即使 1 台坏了，数据还在。

```xml
<?xml version="1.0" encoding="UTF-8"?>
<?xml-stylesheet type="text/xsl" href="configuration.xsl"?>
<configuration>
    <!-- ====== 配置1: NameNode元数据存储目录 ====== -->
    <!-- 元数据 = "文件的目录信息"（哪个文件存在哪台机器上） -->
    <!-- 这个目录非常重要！丢失意味着所有文件找不到 -->
    <property>
        <name>dfs.namenode.name.dir</name>
        <value>file:/opt/hadoop/data/namenode</value>
    </property>

    <!-- ====== 配置2: DataNode数据块存储目录 ====== -->
    <!-- 实际的文件数据块存在这个目录下 -->
    <!-- 建议放在空间大的磁盘上 -->
    <property>
        <name>dfs.datanode.data.dir</name>
        <value>file:/opt/hadoop/data/datanode</value>
    </property>
```

```

    <!-- ====== 配置3: 副本数 (重要!) ====== -->
    <!-- 每个数据块存几份副本 -->
    <!-- 3 = 存3份 (生产环境标准) -->
    <!-- 竞赛中如果只有3台机器, 设3没问题 -->
    <!-- 如果只有1台机器测试, 设1 -->
    <property>
        <name>dfs.replication</name>
        <value>3</value>
    </property>

    <!-- ====== 配置4: SecondaryNameNode地址 ====== -->
    <!-- SecondaryNameNode 帮 NameNode 合并编辑日志 -->
    <!-- 通常放在和 NameNode 不同的机器上 (竞赛中可放同台) -->
    <property>
        <name>dfs.namenode.secondary.http-address</name>
        <value>node1:9868</value>
    </property>
</configuration>
```

4.5.4 yarn-site.xml——配置资源调度器

这个文件是干什么的？

YARN = ”资源调度中心”。它决定”哪个任务用哪台机器的 CPU 和内存”。

类比：就像学校食堂的打饭阿姨——学生（任务）来打饭，阿姨（YARN）告诉学生去哪个窗口（哪台机器）。阿姨自己在 node1（ResourceManager），每个窗口有一个服务员（NodeManager 在 node2、node3）。

```
<?xml version="1.0" encoding="UTF-8"?>
<?xml-stylesheet type="text/xsl" href="configuration.xsl"?>
<configuration>
    <!-- ====== 配置1: ResourceManager在哪 ====== -->
    <!-- ResourceManager 是"总调度员", 放在 node1 -->
    <!-- 所有计算任务都要先向它申请资源 -->
    <property>
        <name>yarn.resourcemanager.hostname</name>
        <value>node1</value>
    </property>
```

```

    <!-- ====== 配置2: NodeManager辅助服务 ====== -->
    <!-- mapreduce_shuffle = MapReduce 的数据洗牌服务 -->
    <!-- 不配这个, MapReduce 任务会报错 -->
    <property>
        <name>yarn.nodemanager.aux-services</name>
        <value>mapreduce_shuffle</value>
    </property>

    <!-- ====== 配置3: 关闭虚拟内存检查 ====== -->
    <!-- Linux 的虚拟内存检查在竞赛环境经常误报 -->
    <!-- 关掉它防止任务被误杀 -->
    <property>
        <name>yarn.nodemanager.vmem-check-enabled</name>
        <value>false</value>
    </property>
</configuration>
```

4.5.5 mapred-site.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<?xml-stylesheet type="text/xsl" href="configuration.xsl"?>
<configuration>
    <!-- 指定MapReduce运行在YARN上 -->
    <property>
        <name>mapreduce.framework.name</name>
        <value>yarn</value>
    </property>
</configuration>
```

4.5.6 workers

```
vi $HADOOP_HOME/etc/hadoop/workers

# 添加DataNode节点
node1
node2
node3
```

4.6 分发 Hadoop 到所有节点

```
# 将配置好的Hadoop分发到node2和node3
scp -r /opt/hadoop-3.3.4 root@node2:/opt/
scp -r /opt/hadoop-3.3.4 root@node3:/opt/

# 在node2和node3上创建软链接并配置环境变量
ln -s /opt/hadoop-3.3.4 /opt/hadoop
```

4.7 格式化 NameNode 并启动集群

格式化只会执行一次

`hdfs namenode -format` 只在第一次部署时执行。重复执行会清空 HDFS 数据！

```
# 【在 node1 上执行】格式化 NameNode（仅第一次）
hdfs namenode -format

# 【在 node1 上执行】启动 HDFS（自动在 node2、node3 启动
  DataNode）
start-dfs.sh

# 【在 node1 上执行】启动 YARN
start-yarn.sh

# 【在 node1 上执行】验证
jps  # 应有 NameNode, SecondaryNameNode, ResourceManager
```

启动后必做的验证

```
# 在 node2、node3 上执行
jps  # 应有 DataNode, NodeManager

# 在 node1 上查看 HDFS 状态
hdfs dfsadmin -report  # 应看到 node2、node3 的 DataNode
```

```

# 访问 Web UI
# HDFS: http://node1:9870
# YARN: http://node1:8088
```

验证 Hadoop 集群

```
# 查看HDFS报告
hdfs dfsadmin -report

# 访问Web UI
# HDFS NameNode: http://node1:9870
# YARN ResourceManager: http://node1:8088
```

5 HDFS 分布式文件系统

5.1 HDFS 是什么？

用拼图理解 HDFS

一个 10GB 的电影文件，你的单个硬盘放不下。

HDFS 的做法：把文件切成很多小块（默认每块 128MB），分散存到不同机器上。

```
10GB电影
    块1 (128MB) --> 存在 node1
    块2 (128MB) --> 存在 node2
    块3 (128MB) --> 存在 node3
    块4 (128MB) --> 存在 node1  <-- 副本
    ...
```

关键设计：

- **分块存储**：大文件切成小块（默认 128MB/块）
- **多副本**：每块存 3 份（默认），即使某台机器坏了数据还在
- **NameNode**：记录”哪个文件的哪个块在哪台机器上”（像图书馆的目录）
- **DataNode**：实际存数据块的机器（像图书馆的书架）

竞赛中你需要做的：在 node1 启动 NameNode，在 node2、node3 启动 DataNode。NameNode 告诉你数据在哪里，DataNode 实际存储数据。

5.2 HDFS 基本概念

HDFS 架构

- **NameNode**：管理文件系统命名空间和客户端访问
- **DataNode**：存储实际的数据块
- **Block**：数据块，默认 128MB

5.3 HDFS 常用命令

表 5: HDFS 常用命令

命令	说明
hdfs dfs -ls /	查看 HDFS 根目录
hdfs dfs -mkdir /path	创建目录
hdfs dfs -put local hdfs	上传文件
hdfs dfs -get hdfs local	下载文件
hdfs dfs -cat /path	查看文件内容
hdfs dfs -rm /path	删除文件
hdfs dfsadmin -report	查看集群状态

动手练习

1. 在 HDFS 上创建目录：`/user/root/data`
2. 上传本地文件到 HDFS
3. 查看上传的文件内容
4. 下载 HDFS 文件到本地

6 Zookeeper 协调服务

6.1 Zookeeper 是什么？

用班级班长理解 Zookeeper

想象一个班级有 30 个同学（30 台服务器）：

- 谁是班长？（选举 Leader）
- 班长不在了，谁来接任？（故障转移）
- 老师的通知怎么让所有人都知道？（配置同步）
- 怎么防止两个人同时抢同一个任务？（分布式锁）

Zookeeper 就是分布式系统的”班长”，它负责：

- 选举一个 Leader（谁说了算）
- 监控其他节点的存活状态（谁在线）
- 存储和同步配置信息（通知所有人）

为什么 Kafka 需要 Zookeeper？ Kafka 用 Zookeeper 来记录”哪些 Broker 在线”、”Topic 的分区在哪个 Broker 上”等元信息。没有 Zookeeper，Kafka 就不知道集群的状态。

6.2 Zookeeper 简介

Zookeeper 作用

Zookeeper 是一个分布式协调服务，为分布式应用提供一致性服务，常用于：

- 配置管理
- 命名服务
- 分布式锁
- 集群管理

6.3 Zookeeper 安装部署

```
# 下载Zookeeper（以3.7.1版本为例）
```

```
wget https://repo.huaweicloud.com/apache/zookeeper/zookeeper
    -3.7.1/apache-zookeeper-3.7.1-bin.tar.gz

# 解压
tar -zxvf apache-zookeeper-3.7.1-bin.tar.gz -C /opt/
ln -s /opt/apache-zookeeper-3.7.1-bin /opt/zookeeper

# 配置环境变量
vi /etc/profile
export ZOOKEEPER_HOME=/opt/zookeeper
export PATH=$ZOOKEEPER_HOME/bin:$PATH
source /etc/profile
```

6.4 Zookeeper 配置

```
# 创建数据目录
mkdir -p /opt/zookeeper/data

# 复制配置文件
cd /opt/zookeeper/conf
cp zoo_sample.cfg zoo.cfg  # 从模板复制
vi zoo.cfg

# ====== 数据存储目录 ======
dataDir=/opt/zookeeper/data

# ====== 客户端连接端口 ======
clientPort=2181

# ====== 集群成员列表 ======
# 格式: server.N=主机名:通信端口:选举端口
# N 必须和 myid 文件内容一致
# 所有节点的配置必须完全相同
server.1=node1:2888:3888
server.2=node2:2888:3888
server.3=node3:2888:3888
```

6.5 配置节点 ID

```
# 在node1上
echo "1" > /opt/zookeeper/data/myid

# 在node2上
echo "2" > /opt/zookeeper/data/myid

# 在node3上
echo "3" > /opt/zookeeper/data/myid
```

6.6 启动 Zookeeper

```
# 在所有节点上启动
zkServer.sh start

# 查看状态
zkServer.sh status

# 停止
zkServer.sh stop
```

7 HBase 分布式数据库

7.1 HBase 是什么？

用通讯录理解 HBase

MySQL 是" 表格型数据库"——每行有固定的列（姓名、年龄、地址）。
HBase 是" 宽列型数据库"——像一个超大通讯录：

- 每个人（Row Key）可以有不同数量的信息
- 张三只有手机号，李四有手机号 + 邮箱 + 微信
- 你不需要提前定义" 每个人必须有几列"

什么时候用 HBase？

- 数据量很大（上亿条记录）

- 需要按某个关键词（Row Key）快速查找
- 列的数量不固定

类比：MySQL 像 Excel 表格，HBase 像字典（按关键词查内容）。

7.2 HBase 简介

HBase 特点

HBase 是一个分布式、可扩展、面向列的 NoSQL 数据库，基于 HDFS 存储，提供高吞吐量的随机读写能力。

核心组件：

- **HMaster**：管理 RegionServer，负责表的 DDL 操作
- **RegionServer**：处理客户端读写请求，管理 Region
- **Zookeeper**：协调 HMaster 和 RegionServer

7.3 HBase 安装部署

```
# 下载HBase（以2.4.15版本为例）
wget https://repo.huaweicloud.com/apache/hbase/2.4.15/hbase
    -2.4.15-bin.tar.gz

# 解压
tar -zxvf hbase-2.4.15-bin.tar.gz -C /opt/
ln -s /opt/hbase-2.4.15 /opt/hbase

# 配置环境变量
vi /etc/profile
export HBASE_HOME=/opt/hbase
export PATH=$HBASE_HOME/bin:$PATH
source /etc/profile
```

7.4 HBase 配置

7.4.1 hbase-env.sh

```
vi $HBASE_HOME/conf/hbase-env.sh

# 添加以下内容
export JAVA_HOME=/opt/jdk1.8.0_212
export HBASE_MANAGES_ZK=false  # 使用外部Zookeeper
```

7.4.2 hbase-site.xml

```
<?xml version="1.0"?>
<?xml-stylesheet type="text/xsl" href="configuration.xsl"?>
<configuration>
    <!-- 指定HDFS存储路径 -->
    <property>
        <name>hbase.rootdir</name>
        <value>hdfs://node1:9000/hbase</value>
    </property>

    <!-- 启用分布式模式 -->
    <property>
        <name>hbase.cluster.distributed</name>
        <value>true</value>
    </property>

    <!-- Zookeeper集群地址 -->
    <property>
        <name>hbase.zookeeper.quorum</name>
        <value>node1,node2,node3</value>
    </property>
</configuration>
```

7.5 启动 HBase

```
# 在node1上启动HBase
start-hbase.sh

# 查看进程
jps
```

```
# 应该看到 HMaster 和 HRegionServer

# 停止HBase
stop-hbase.sh
```

7.6 HBase Shell 操作

```
# 进入HBase Shell
hbase shell

# 常用命令
help                          # 查看帮助
list                          # 列出所有表
create 'student', 'info', 'score'     # 创建表
describe 'student'                    # 查看表结构
put 'student', '1001', 'info:name', '张三'  # 插入数据
get 'student', '1001'                 # 查询单行
scan 'student'                        # 扫描全表
count 'student'                       # 统计行数
disable 'student'                     # 禁用表
drop 'student'                        # 删除表
exit                                  # 退出
```

8 Hive 数据仓库

8.1 Hive 是什么？

用翻译器理解 Hive

HDFS 上存的数据是”裸文件”（文本、JSON），没有表结构，不能用 SQL 查询。**Hive 的作用**：给 HDFS 上的文件”翻译”成表格，让你用 SQL 查询。

```
HDFS 文件（裸数据）   --Hive翻译-->  表格（可以用SQL查）
/user/data/log.txt                  SELECT * FROM log WHERE ...
```

类比：

- HDFS 是仓库里的散装货物

- Hive 是货架标签系统——告诉你”第 3 排第 2 层是方便面”
- 你不用搬货，只需要查标签（SQL 查询）就知道有什么

Hive 不是数据库！它只是一个”翻译器”，数据还是存在 HDFS 上。所以 Hive 查询速度不如 MySQL 快，但能处理海量数据。

8.2 Hive 简介

Hive 是什么？

Hive 是基于 Hadoop 的数据仓库工具，可以将结构化数据文件映射为数据库表，并提供类 SQL 查询（HQL）。

8.3 Hive 安装部署

```
# 下载Hive（以3.1.3版本为例）
wget https://repo.huaweicloud.com/apache/hive/hive-3.1.3/apache-hive-3.1.3-bin.tar.gz

# 解压
tar -zxvf apache-hive-3.1.3-bin.tar.gz -C /opt/
ln -s /opt/apache-hive-3.1.3-bin /opt/hive

# 配置环境变量
vi /etc/profile
export HIVE_HOME=/opt/hive
export PATH=$HIVE_HOME/bin:$PATH
source /etc/profile
```

8.4 安装 MySQL（元数据存储）

```
# 安装MySQL
yum install -y mysql-server

# 启动MySQL
systemctl start mysqld
systemctl enable mysqld
```

```

# 修改密码
mysqladmin -u root password '123456'

# 登录MySQL创建Hive元数据库
mysql -uroot -p123456

# 执行SQL
CREATE DATABASE hive DEFAULT CHARACTER SET utf8;
CREATE USER 'hive'@'%' IDENTIFIED BY '123456';
GRANT ALL PRIVILEGES ON hive.* TO 'hive'@'%';
FLUSH PRIVILEGES;
EXIT;
```

8.5 Hive 配置

8.5.1 hive-site.xml

```xml
<?xml version="1.0" encoding="UTF-8"?>
<?xml-stylesheet type="text/xsl" href="configuration.xsl"?>
<configuration>
    <!-- MySQL连接配置 -->
    <property>
        <name>javax.jdo.option.ConnectionURL</name>
        <value>jdbc:mysql://node1:3306/hive?
            createDatabaseIfNotExist=true&amp;useSSL=false</value
            >
    </property>

    <property>
        <name>javax.jdo.option.ConnectionDriverName</name>
        <value>com.mysql.jdbc.Driver</value>
    </property>

    <property>
        <name>javax.jdo.option.ConnectionUserName</name>
        <value>hive</value>
    </property>

```

```
    <property>
        <name>javax.jdo.option.ConnectionPassword</name>
        <value>123456</value>
    </property>
</configuration>
```

8.6 初始化 Hive 元数据

```
# 上传MySQL驱动到 $HIVE_HOME/lib/

# 初始化元数据
schematool -dbType mysql -initSchema
```

8.7 启动 Hive

```
# 启动Hive CLI
hive

# 或者启动HiveServer2（推荐）
nohup hiveserver2 > /var/log/hive/hiveserver2.log 2>&1 &

# 使用Beeline连接
beeline -u jdbc:hive2://node1:10000 -n root
```

9 Spark 计算引擎

9.1 Spark 是什么？

用超级计算器理解 Spark

Hadoop 的 MapReduce 计算方式很慢——每次计算都要把中间结果写到硬盘上，再读出来继续算。

Spark 的改进：把中间结果放在内存里，不用反复读写硬盘，速度提升 10-100 倍。

类比：

- MapReduce = 做作业时每算一步都抄到本子上，再翻出来继续算

- Spark = 做作业时直接在草稿纸上算，算完再抄答案

Spark 能做什么？

- **批处理**：处理大量历史数据（比如统计一年的销售总额）
- **流处理**：实时处理数据流（比如实时监控今天的销售趋势）
- **SQL 查询**：用 SQL 语句查询大数据
- **机器学习**：训练预测模型

竞赛中你需要做的：在 node1 上配置 Spark Master，node2、node3 上配置 Spark Worker，然后提交任务让它们协同计算。

9.2 Spark 简介

Spark 特点

Spark 是一个快速、通用、可扩展的大数据计算引擎，支持批处理、流处理、SQL 查询、机器学习和图计算。

9.3 Spark 安装部署

```
# 下载Spark（以3.3.2版本为例，预编译Hadoop版本）
wget https://repo.huaweicloud.com/apache/spark/spark-3.3.2/spark-3.3.2-bin-hadoop3.tgz

# 解压
tar -zxvf spark-3.3.2-bin-hadoop3.tgz -C /opt/
ln -s /opt/spark-3.3.2-bin-hadoop3 /opt/spark

# 配置环境变量
vi /etc/profile
export SPARK_HOME=/opt/spark
export PATH=$SPARK_HOME/bin:$SPARK_HOME/sbin:$PATH
source /etc/profile
```

9.4 Spark 配置

9.4.1 spark-env.sh

```
cd $SPARK_HOME/conf
cp spark-env.sh.template spark-env.sh

vi spark-env.sh

# 添加以下内容
export JAVA_HOME=/opt/jdk1.8.0_212
export HADOOP_HOME=/opt/hadoop
export HADOOP_CONF_DIR=/opt/hadoop/etc/hadoop
export SPARK_MASTER_HOST=node1
export SPARK_MASTER_PORT=7077
export SPARK_WORKER_CORES=2
export SPARK_WORKER_MEMORY=4g
```

9.4.2 workers

```
cp workers.template workers

vi workers

# 添加Worker节点
node1
node2
node3
```

9.5 启动 Spark 集群

```
# 启动Master和Workers
start-all.sh

# 启动History Server
start-history-server.sh

# 查看进程
```

```
jps

# 停止集群
stop-all.sh
```

10 Kafka 消息队列

10.1 Kafka 是什么？

用传送带理解 Kafka

想象一个工厂流水线：

- 工人 A（Flume）把产品放到传送带上
- 传送带（Kafka）暂时存储产品
- 工人 B（Spark）从传送带上取产品加工

为什么要用 Kafka？

- **速度不匹配**：A 生产速度快，B 处理速度慢。Kafka 做缓冲区，防止 B 来不及处理而丢数据
- **解耦**：A 和 B 不需要直接通信。A 只管往 Kafka 放，B 只管从 Kafka 取
- **容错**：如果 B 挂了，数据还在 Kafka 里，B 恢复后可以继续取

Kafka 的核心概念：

- **Topic**：消息分类（比如”用户行为”是一个 Topic，”订单”是另一个 Topic）
- **Partition**：分区，Topic 的子单元，可以让多个消费者并行读取
- **Broker**：Kafka 服务器，存储消息
- **Producer**：消息生产者（Flume 就是 Producer）
- **Consumer**：消息消费者（Spark 就是 Consumer）

10.2 Kafka 简介

Kafka 作用

Kafka 是一个分布式流处理平台，主要用于：

- 消息队列：发布和订阅消息流
- 存储：持久化存储消息流
- 处理：实时处理消息流

10.3 Kafka 安装部署

```
# 下载Kafka（以3.4.0版本为例）
wget https://repo.huaweicloud.com/apache/kafka/3.4.0/kafka_2
    .12-3.4.0.tgz

# 解压
tar -zxvf kafka_2.12-3.4.0.tgz -C /opt/
ln -s /opt/kafka_2.12-3.4.0 /opt/kafka

# 配置环境变量
vi /etc/profile
export KAFKA_HOME=/opt/kafka
export PATH=$KAFKA_HOME/bin:$PATH
source /etc/profile
```

10.4 Kafka 配置

Kafka 配置详解

Kafka 只装在 node1（单 Broker）。
关键配置：

- `broker.id`：Broker 编号，每个 Broker 必须不同
- `listeners`：监听地址（`0.0.0.0` = 所有网卡）
- `advertised.listeners`：对外公布地址（**不能写 0.0.0.0**）
- `zookeeper.connect`：Zookeeper 集群地址

server.properties 是干什么的？

server.properties 是 Kafka Broker（消息服务器）的配置文件。它告诉 Kafka：

- 自己是谁（broker.id）
- 在哪个端口接收消息（listeners）
- 数据存在哪里（log.dirs）
- Zookeeper 在哪（zookeeper.connect）

```
# 【在 node1 上执行】配置 Kafka
vi $KAFKA_HOME/config/server.properties

# ====== 配置1：Broker编号 ======
# 每个 Kafka 服务器必须有唯一编号
# 竞赛中只有1个 Broker，设0即可
broker.id=0

# ====== 配置2：监听地址 ======
# listeners：Kafka 在哪个地址和端口接收数据
# 0.0.0.0 表示监听所有网卡（本机的所有IP都能访问）
listeners=PLAINTEXT://0.0.0.0:9092

# ====== 配置3：对外公布地址（非常重要！） ======
# 客户端（Flume、Spark）用这个地址连接 Kafka
# 必须写具体的主机名或IP，绝对不能写 0.0.0.0！
advertised.listeners=PLAINTEXT://node1:9092

# ====== 配置4：消息存储目录 ======
# Kafka 的所有消息存在这个目录下
log.dirs=/opt/kafka-logs

# ====== 配置5：Zookeeper地址 ======
# Kafka 依赖 Zookeeper 来管理集群信息
# 写 Zookeeper 集群所有节点的地址
zookeeper.connect=node1:2181,node2:2181,node3:2181

# ====== 配置6：默认分区数 ======
# 每个 Topic 默认分成几个分区
```

```
# 分区越多，并行度越高
num.partitions=3
```

重要

advertised.listeners 必须写 node1（或实际 IP），**绝对不能写 0.0.0.0**！否则客户端无法连接。

10.5 启动 Kafka

```
# 在所有节点上启动（先启动Zookeeper）
zkServer.sh start

# 启动Kafka（每个节点）
kafka-server-start.sh -daemon $KAFKA_HOME/config/server.properties

# 停止Kafka
kafka-server-stop.sh
```

11 Flume 数据采集

11.1 Flume 是什么？

用搬家公司理解 Flume

你的数据在某个地方（比如日志文件），你需要把它搬到另一个地方（比如 Kafka）。**Flume 就是"数据搬家公司"**，它的工作流程：

```
[日志文件] --Source读取--> [Channel暂存] --Sink发送--> [Kafka]
```

三个核心组件（记住"搬、存、发"）：

- **Source**（搬）：从哪里读数据——可以监控日志文件、监听端口等
- **Channel**（存）：暂存数据——用内存（快）或文件（安全）
- **Sink**（发）：发到哪里——可以发到 Kafka、HDFS、HBase 等

为什么不用简单的命令？比如 `cat file | kafka-producer` 不行吗？
在生产环境中，Flume 提供了：

- 断点续传（程序挂了不丢数据）
- 批量发送（提高效率）
- 容错机制（自动重试失败的发送）
- 可配置的数据转换（过滤、格式化）

11.2 Flume 简介

Flume 架构

Flume 是一个分布式、可靠、高可用的海量日志采集、聚合和传输系统。

核心组件：

- **Agent**：Flume 运行的最小单位，包含 Source、Channel、Sink
- **Source**：数据源，负责收集数据
- **Channel**：数据通道，临时存储数据
- **Sink**：数据目的地，负责输出数据

11.3 Flume 安装部署

```
# 下载Flume（以1.11.0版本为例）
wget https://repo.huaweicloud.com/apache/flume/1.11.0/apache-flume-1.11.0-bin.tar.gz

# 解压
tar -zxvf apache-flume-1.11.0-bin.tar.gz -C /opt/
ln -s /opt/apache-flume-1.11.0-bin /opt/flume

# 配置环境变量
vi /etc/profile
export FLUME_HOME=/opt/flume
export PATH=$FLUME_HOME/bin:$PATH
source /etc/profile
```

11.4 配置采集任务

```
vi $FLUME_HOME/conf/log_to_kafka.conf

# 添加以下内容
agent.sources = src
agent.channels = ch
agent.sinks = sink

agent.sources.src.type = exec
agent.sources.src.command = tail -F /var/log/test.log
agent.sources.src.channels = ch

agent.channels.ch.type = memory
agent.channels.ch.capacity = 10000

agent.sinks.sink.type = org.apache.flume.sink.kafka.KafkaSink
agent.sinks.sink.kafka.bootstrap.servers = node1:9092
agent.sinks.sink.kafka.topic = log-topic
agent.sinks.sink.channel = ch
```

12 ClickHouse 列式数据库

12.1 ClickHouse 是什么？

用超级 Excel 理解 ClickHouse

MySQL 是”行式数据库”——查一行数据很快，但统计分析很慢。

ClickHouse 是”列式数据库”——统计分析极快。

类比：

- MySQL（行式）= 一张写满数据的纸，你只想看”成绩”这一列，但必须从头到尾扫一遍
- ClickHouse（列式）= 每列单独一张纸，你想看”成绩”直接翻那一张

什么时候用 ClickHouse？

- 需要做大量统计分析（COUNT、SUM、AVG、GROUP BY）
- 数据量巨大（上亿行）
- 对查询速度要求高

ClickHouse vs Hive：

- Hive：数据量无限，查询慢（适合离线分析）
- ClickHouse：数据量有限（单机能存几百亿行），查询极快（适合实时分析）

12.2 ClickHouse 简介

ClickHouse 特点

ClickHouse 是一个用于联机分析 (OLAP) 的列式数据库管理系统，具有高性能、高压缩比、实时分析等特点。

12.3 ClickHouse 安装部署

```
# 安装依赖
yum install -y yum-utils

# 添加ClickHouse仓库
yum-config-manager --add-repo https://packages.clickhouse.com/
    rpm/clickhouse.repo

# 安装ClickHouse
yum install -y clickhouse-server clickhouse-client

# 启动服务
systemctl start clickhouse-server
systemctl enable clickhouse-server
```

12.4 ClickHouse 客户端操作

```
# 连接ClickHouse
clickhouse-client

# 常用命令
SHOW DATABASES;
CREATE DATABASE test_db;
USE test_db;
```

```
SHOW TABLES;

-- 创建表
CREATE TABLE test_table (
    id UInt32,
    name String,
    age UInt8
) ENGINE = MergeTree()
ORDER BY id;

-- 插入数据
INSERT INTO test_table VALUES (1, 'Alice', 25);

-- 查询数据
SELECT * FROM test_table;
```

13 平台联调与测试

13.1 组件启动顺序

启动口诀：ZK 先，Hadoop 次，Kafka 后

Zookeeper 必须在 Kafka 之前启动，HDFS 必须在 HBase 之前启动。

```
# ===== 第1步：Zookeeper（所有3个节点都执行）=====
zkServer.sh start
# 验证：任意节点执行 zkServer.sh status，应看到 leader/follower

# ===== 第2步：Hadoop（在 node1 上执行）=====
start-dfs.sh
start-yarn.sh
# 验证：jps 检查各节点

# ===== 第3步：HBase（在 node1 上执行）=====
start-hbase.sh

# ===== 第4步：Hive（在 node1 上执行）=====
nohup hiveserver2 > /var/log/hive/hiveserver2.log 2>&1 &

```

```
# ===== 第5步: Spark (在 node1 上执行) =====
start-all.sh
start-history-server.sh

# ===== 第6步: Kafka (在 node1 上执行) =====
# 注意: 必须等 Zookeeper 完全启动后再启动 Kafka
export KAFKA_HEAP_OPTS='-Xmx256M -Xms256M'
kafka-server-start.sh -daemon $KAFKA_HOME/config/server.
    properties
sleep 15  # 等待 Kafka 完全启动

kafka-topics.sh --create --bootstrap-server node1:9092 \
    --replication-factor 1 --partitions 3 --topic user-behavior

# ===== 第7步: ClickHouse (在 node2 上执行) =====
systemctl start clickhouse-server
```

13.2 组件停止顺序

```
# 1. 停止Flume
pkill -f flume

# 2. 停止Kafka
kafka-server-stop.sh

# 3. 停止Spark
stop-history-server.sh
stop-all.sh

# 4. 停止HiveServer2
pkill -f HiveServer2

# 5. 停止HBase
stop-hbase.sh

# 6. 停止Hadoop
stop-all.sh

```

```
# 7. 停止Zookeeper
zkServer.sh stop

# 8. 停止ClickHouse
systemctl stop clickhouse-server
```

14 竞赛模拟实战

14.1 实战场景：电商实时数据分析平台

场景描述

构建一个电商实时数据分析平台，实现：

1. 使用 Flume 采集用户行为日志
2. 通过 Kafka 进行消息传输
3. 使用 Spark Streaming 进行实时处理
4. 将结果存储到 HBase 和 ClickHouse
5. 使用 Hive 进行离线分析

14.2 步骤 1：云端数据仿真平台

数据来源方案

使用云端部署的电商数据仿真平台实时生成数据，平台提供 RESTful API 接口，可直接供 Flume 采集或 Kafka 消费。

数据格式：JSON 格式，包含以下字段：

- timestamp：事件发生时间
- user_id：用户 ID（U10001-U99999）
- event_type：事件类型（view/click/add_cart/buy/search/collect/share）
- session_id：会话 ID（32 位 MD5）
- device：设备类型（iOS/Android/PC/小程序/H5）
- ip：用户 IP 地址

- product_id：商品 ID（P10001-P99999）
- category：商品类别（15 个品类）
- price：商品价格（根据品类自动生成）
- page_source：来源页面
- stay_time：停留秒数

14.2.1 访问仿真平台

平台访问

- 平台地址：`http://163.7.1.176:8889`
- 平台自动实时生成电商用户行为数据（每秒 1 条）
- 内置 1000 个用户画像、500 个商品、15 个品类
- 所有接口均可直接访问，无需鉴权

14.2.2 API 接口说明

表 6: 仿真平台 API 接口

方法	接口路径	说明
GET	/api/data?n=10	生成 N 条数据
GET	/api/log?lines=10	模拟日志输出
GET	/api/history	历史数据查询
GET	/api/history?event_type=buy	按事件筛选
GET	/api/history?category= 手机数码	按品类筛选
GET	/api/users	用户画像
GET	/api/products	商品数据
GET	/api/stats	统计汇总

14.2.3 创建日志文件供 Flume 采集

```
# 创建日志目录
mkdir -p /var/log/ecommerce

# 使用 API 生成数据并保存到本地日志文件
# 方法一：curl 单次获取
```

```
curl -s "http://163.7.1.176:8889/api/data?n=100" | python3 -m
    json.tool > /var/log/ecommerce/user_behavior.log

# 方法二：持续采集（每秒从 API 获取1条，写入日志文件）
while true; do
    curl -s "http://163.7.1.176:8889/api/log?lines=1" >> /var/
        log/ecommerce/user_behavior.log
    sleep 1
done
```

动手练习

1. 访问仿真平台首页，查看实时数据流
2. 使用 curl 获取 10 条数据：`curl ".../api/data?n=10"`
3. 查看用户画像数据：`curl ".../api/users?page=1"`
4. 查看统计汇总：`curl ".../api/stats"`
5. 创建本地日志文件，为后续 Flume 采集做准备

14.3 步骤 2：创建 Kafka Topic

```
# 创建Topic，3个副本，3个分区
kafka-topics.sh --create --bootstrap-server node1:9092 \
    --replication-factor 3 --partitions 3 --topic user-behavior

# 验证Topic创建成功
kafka-topics.sh --describe --bootstrap-server node1:9092 --topic
    user-behavior
```

14.4 步骤 3：创建 HBase 表

```
# 进入HBase Shell
hbase shell

# 创建用户行为表
create 'user_behavior', 'info'
```

```

# 验证表创建成功
list
```

14.5 步骤 4：创建 ClickHouse 表

```
# 连接ClickHouse
clickhouse-client

# 创建电商数据库
CREATE DATABASE IF NOT EXISTS ecommerce;

# 创建用户统计表
CREATE TABLE ecommerce.user_stats (
    user_id String,
    event_type String,
    event_count UInt32,
    stats_date Date
) ENGINE = MergeTree()
ORDER BY (user_id, event_type, stats_date);

# 退出
exit
```

14.6 步骤 5：配置并启动 Flume 采集

```
# 编辑Flume配置文件
vi $FLUME_HOME/conf/ecommerce.conf
```

配置文件内容：

```
agent.sources = src
agent.channels = ch
agent.sinks = kafka

# Source配置：监控日志文件
agent.sources.src.type = exec
```

```
agent.sources.src.command = tail -F /var/log/ecommerce/
    user_behavior.log
agent.sources.src.channels = ch

# Channel配置：内存通道
agent.channels.ch.type = memory
agent.channels.ch.capacity = 10000
agent.channels.ch.transactionCapacity = 1000

# Sink配置：输出到Kafka
agent.sinks.kafka.type = org.apache.flume.sink.kafka.KafkaSink
agent.sinks.kafka.kafka.bootstrap.servers = node1:9092,node2
    :9092,node3:9092
agent.sinks.kafka.kafka.topic = user-behavior
agent.sinks.kafka.kafka.flumeBatchSize = 100
agent.sinks.kafka.channel = ch
```

启动 Flume：

```
# 启动Flume采集
flume-ng agent --name agent --conf $FLUME_HOME/conf \
    --conf-file $FLUME_HOME/conf/ecommerce.conf \
    -Dflume.root.logger=INFO,console
```

14.7 步骤 6：编写 Spark Streaming 处理程序

创建处理程序文件：

```
vi /opt/spark-apps/ecommerce_analysis.py
```

程序内容：

```
from pyspark.sql import SparkSession
from pyspark.sql.functions import *
from pyspark.sql.types import *

# 创建SparkSession
spark = SparkSession.builder \
    .appName("Ecommerce Analysis") \
    .master("spark://node1:7077") \
```

```python
    .getOrCreate()

# 定义JSON schema
schema = StructType([
    StructField("timestamp", StringType(), True),
    StructField("user_id", StringType(), True),
    StructField("event_type", StringType(), True),
    StructField("session_id", StringType(), True),
    StructField("device", StringType(), True),
    StructField("ip", StringType(), True),
    StructField("product_id", StringType(), True),
    StructField("category", StringType(), True),
    StructField("price", DoubleType(), True),
    StructField("page_source", StringType(), True),
    StructField("stay_time", IntegerType(), True)
])

# 读取Kafka数据
df = spark.readStream \
    .format("kafka") \
    .option("kafka.bootstrap.servers", "node1:9092,node2:9092,node3:9092") \
    .option("subscribe", "user-behavior") \
    .load()

# 解析JSON数据
parsed = df.select(
    from_json(col("value").cast("string"), schema).alias("data")
)

# 统计用户行为
event_stats = parsed.groupBy("data.user_id", "data.event_type") \
    .count() \
    .withColumnRenamed("count", "event_count")

# 写入HBase
event_stats.writeStream \
    .outputMode("update") \
    .format("org.apache.hadoop.hbase.spark") \
```

```
    .option("hbase.table", "user_behavior") \
    .option("hbase.spark.use.hbasecontext", "false") \
    .start()

# 写入ClickHouse
event_stats.writeStream \
    .outputMode("update") \
    .format("jdbc") \
    .option("url", "jdbc:clickhouse://node1:8123/ecommerce") \
    .option("dbtable", "user_stats") \
    .option("user", "default") \
    .option("driver", "com.clickhouse.jdbc.ClickHouseDriver") \
    .start()

# 等待流处理
spark.streams.awaitAnyTermination()
```

提交 Spark 作业:

```
# 提交Spark Streaming作业
spark-submit --master spark://node1:7077 \
    --packages org.apache.spark:spark-sql-kafka-0-10_2.12:3.3.2 \
    /opt/spark-apps/ecommerce_analysis.py
```

14.8 步骤 7: 验证数据流转

14.8.1 验证 Kafka 数据

```
# 消费Kafka数据, 查看是否采集成功
kafka-console-consumer.sh --bootstrap-server node1:9092 \
    --topic user-behavior --from-beginning
```

14.8.2 验证 HBase 数据

```
# 进入HBase Shell
hbase shell

```

```
# 扫描用户行为表
scan 'user_behavior', {LIMIT=>10}

# 统计行数
count 'user_behavior'
```

14.8.3 验证 ClickHouse 数据

```
# 查询ClickHouse统计结果
clickhouse-client -q "SELECT * FROM ecommerce.user_stats LIMIT 10"

# 按事件类型统计
clickhouse-client -q "SELECT event_type, SUM(event_count) FROM ecommerce.user_stats GROUP BY event_type"
```

14.8.4 验证 Hive 数据

```
# 创建Hive外部表映射HBase数据
hive -e "
CREATE EXTERNAL TABLE user_behavior_hbase (
    rowkey STRING,
    user_id STRING,
    event_type STRING,
    event_count INT
)
STORED BY 'org.apache.hadoop.hive.hbase.HBaseStorageHandler'
WITH SERDEPROPERTIES ('hbase.columns.mapping' = ':key,info:user_id,info:event_type,info:event_count')
TBLPROPERTIES ('hbase.table.name' = 'user_behavior');

SELECT * FROM user_behavior_hbase LIMIT 10;
"
```

完整流程总结

1. 数据生成器持续产生用户行为日志
2. Flume 监控日志文件，实时采集到 Kafka
3. Spark Streaming 从 Kafka 消费数据，进行实时处理
4. 处理结果同时写入 HBase 和 ClickHouse
5. 使用 Hive 可以对 HBase 数据进行离线分析

15 常用命令速查表

表 7: Hadoop 命令

命令	说明
start-all.sh	启动 Hadoop 集群
stop-all.sh	停止 Hadoop 集群
hdfs dfs -ls /	查看 HDFS 根目录
hdfs dfs -mkdir /path	创建目录
hdfs dfs -put local hdfs	上传文件
hdfs dfsadmin -report	查看集群状态

表 8: HBase 命令

命令	说明
start-hbase.sh	启动 HBase
hbase shell	进入 HBase Shell
list	列出表
create 't', 'cf'	创建表
put 't', 'r', 'cf:c', 'v'	插入数据
get 't', 'r'	查询数据
scan 't'	扫描表

表 9: Hive 命令

命令	说明
hive	启动 Hive CLI
hiveserver2	启动 HiveServer2
show databases	查看数据库
use db	切换数据库
show tables	查看表
select * from t	查询数据

表 10: Spark 命令

命令	说明
start-all.sh	启动 Spark 集群
spark-shell	启动 Scala Shell
pyspark	启动 Python Shell
spark-submit	提交应用
spark-sql	启动 SQL Shell

表 11: Kafka 命令

命令	说明
kafka-server-start.sh	启动 Kafka
kafka-topics.sh –create	创建 Topic
kafka-topics.sh –list	列出 Topic
kafka-console-producer.sh	启动生产者
kafka-console-consumer.sh	启动消费者

16 性能调优与日志管理

16.1 基础性能调优

16.1.1 HDFS 块大小

```
# 修改 hdfs-site.xml 中的 dfs.blocksize
# 默认 128MB，大文件（视频、日志）可设为 256MB
```

```
hdfs dfs -D dfs.blocksize=268435456 -put largefile.dat /user/
    data/
```

HDFS 块大小选择

- 默认 128MB 适合大多数场景
- 大文件（>1GB）设为 256MB 减少元数据开销
- 小文件（<10MB）合并后上传，减少 NameNode 压力

16.1.2 YARN 内存配置

```
# yarn-site.xml 关键参数
yarn.nodemanager.resource.memory-mb=8192     # 节点可用内存
yarn.scheduler.minimum-allocation-mb=1024    # 最小容器内存
yarn.nodemanager.vmem-check-enabled=false    # 关闭虚拟内存检查
    (竞赛环境)
```

16.1.3 Spark Executor 配置

```
# 推荐配置 (8核16GB环境)
spark-submit --master yarn \
    --num-executors 4 \
    --executor-cores 2 \
    --executor-memory 4g \
    --driver-memory 2g \
    your_app.py
```

Spark Executor 最佳实践

- Executor 数量 = 节点数 × 每节点 Executor 数
- 每 Executor 核心数建议 2-5
- Executor 内存建议 4-8GB（留 1GB 给系统）

16.1.4 Kafka 分区与消费者

```
# 创建 Topic 时设置分区数（建议 = 消费者线程数）
kafka-topics.sh --create --bootstrap-server node1:9092 \
    --replication-factor 3 --partitions 6 --topic user-behavior
```

16.2 组件日志路径速查表

表 12: 各组件日志文件路径

组件	日志路径
HDFS	`$HADOOP/logs/hadoop-*.log`
YARN	`$HADOOP/logs/yarn-*.log`
Zookeeper	`$ZK/logs/zookeeper.out`
HBase	`$HBASE/logs/hbase-*.log`
Hive	`$HIVE/logs/hive.log`
Spark	`$SPARK/logs/spark-*.log`
Kafka	`$KAFKA/logs/server.log`
Flume	控制台输出或指定文件
ClickHouse	`/var/log/clickhouse-server/`

排查问题的方法

1. 检查服务进程是否在运行：`jps` 或 `ps aux | grep 组件名`
2. 查看对应日志文件的最后几行：`tail -100 $LOG_FILE`
3. 检查端口是否被占用：`netstat -anp | grep 端口号`
4. 检查网络连通性：`ping node2`，`telnet node2 2181`
5. 检查防火墙和 SELinux 是否关闭

17 常见问题排查

问题 1：Hadoop 启动失败

现象：NameNode 或 DataNode 无法启动

解决方案：

```
# 检查日志
cat $HADOOP_HOME/logs/hadoop-root-namenode-node1.log

```

```
# 格式化NameNode（注意：会清空数据）
hdfs namenode -format

# 检查端口占用
netstat -anp | grep 9000
```

问题 2：HBase 连接 Zookeeper 失败

现象：HMaster 无法启动

解决方案：

```
# 检查Zookeeper状态
zkServer.sh status

# 检查hbase-site.xml配置
cat $HBASE_HOME/conf/hbase-site.xml

# 检查HDFS上HBase目录
hdfs dfs -ls /hbase
```

如果从 PDF 复制 URL 有问题

从 PDF 中复制的长 URL 可能会有空格。解决办法：

1. 复制后粘贴到终端，手动删除 URL 中的空格
2. 或使用本书提供的安装脚本（见附录 B）
3. 或在浏览器中打开 URL 后从地址栏复制

故障排查决策树——按图索骥找问题

当某个组件启动失败时，按以下顺序排查：

```
服务启动失败？
  |
  |-- jps 看进程是否在运行
  |     |-- 不在 → 看日志（cat $组件/logs/*.log | tail -50）
  |     |        |-- 日志报 "Connection refused" → 检查依赖服务是否启动
  |     |        |-- 日志报 "OutOfMemory" → 减小堆内存
  |     |        |-- 日志报 "Permission denied" → 检查用户权限
```

```
|     |        |-- 日志报 "Address already in use" → 端口被占用，换端口或杀进程
|     |-- 在 → 服务已启动，检查下一步
|
|-- 客户端连不上？
|     |-- ping 目标主机名 → 检查网络
|     |-- 检查 /etc/hosts 是否一致（三台机器都要检查）
|     |-- 检查防火墙（systemctl status firewalld）
|     |-- 检查 advertised.listeners（Kafka 特有）
|
|-- 数据读不到？
      |-- HDFS: hdfs dfs -ls /  看文件是否存在
      |-- Hive: show tables; 看表是否创建
      |-- Kafka: kafka-topics.sh --list 看 Topic 是否存在
```

学习建议

1. **循序渐进**：按照章节顺序逐步部署，确保每个组件正常运行后再进行下一步
2. **理解原理**：不要只记命令，要理解每个组件的作用和工作原理
3. **动手实践**：多练习各种操作命令，熟悉各个组件的使用
4. **查看日志**：遇到问题时首先查看日志文件，定位问题原因
5. **联调测试**：完成部署后一定要进行数据流转测试，确保各组件协同工作

18 附录 B：一键下载安装脚本

使用方法

将以下脚本保存为 download.sh，在 node1 上执行 bash download.sh，即可自动下载所有安装包。

```bash
#!/bin/bash
# 一键下载所有大数据组件安装包
# 在 node1 上执行：bash download.sh

mkdir -p /opt/bigdata-pkg && cd /opt/bigdata-pkg

echo "=== 下载 Hadoop ==="
wget https://repo.huaweicloud.com/apache/hadoop/common/hadoop
    -3.3.4/hadoop-3.3.4.tar.gz
```

```

echo "=== 下载 Zookeeper ==="
wget https://repo.huaweicloud.com/apache/zookeeper/zookeeper-3.7.1/apache-zookeeper-3.7.1-bin.tar.gz

echo "=== 下载 HBase ==="
wget https://repo.huaweicloud.com/apache/hbase/2.4.15/hbase-2.4.15-bin.tar.gz

echo "=== 下载 Hive ==="
wget https://repo.huaweicloud.com/apache/hive/hive-3.1.3/apache-hive-3.1.3-bin.tar.gz

echo "=== 下载 Spark ==="
wget https://repo.huaweicloud.com/apache/spark/spark-3.3.2/spark-3.3.2-bin-hadoop3.tgz

echo "=== 下载 Kafka ==="
wget https://repo.huaweicloud.com/apache/kafka/3.4.0/kafka_2.12-3.4.0.tgz

echo "=== 下载 Flume ==="
wget https://repo.huaweicloud.com/apache/flume/1.11.0/apache-flume-1.11.0-bin.tar.gz

echo "=== 全部下载完成 ==="
ls -lh /opt/bigdata-pkg/
```

下载完成后

1. 将安装包分发到 node2、node3：`scp *.tar.gz node2:/opt/`
2. 在每台机器上解压安装（见各组件章节）